

Assignment - 19

1] Diff betⁿ malloc() and new, free and delete
malloc() / free() [C] new / delete [C++]

memory
type
initialization
Deallocation
Type Safety

Allocates raw memory.
Doesn't initialize memory.
free(ptr);
Needs explicit typecasting

Syntax
Performance

ptr = (int *) malloc (sizeof(int));
Relatively low overhead.

Error
Handling

Returns 'NULL' on failure

Allocates memory & auto constructs
initialize objects during allocation
delete ptr; (calls destructor)
Automatically provides type
safety.

ptr = new int;
May have higher overhead
for constructors.
Throws std::bad_alloc
exception on failure

Program to Allocate Memory for 10 integer using malloc().

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int *) malloc (10 * sizeof (int));
    if (arr == NULL) {
        printf("memory allocation failed. \n");
        return 1;
    }

    for (int i = 0; i < 10; i++) {
        arr[i] = i + 1;
        printf("%d", arr[i]);
    }

    free (arr); // Deallocate memory
    return 0;
}
```

Program to Allocate memory for 10 integers using realloc().

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int *) malloc (5 * sizeof (int));
    if (arr == NULL) {
        printf("memory allocation failed. \n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        arr[i] = i + 1;
    }
}
```


Date _____
Page _____

```
arr = (int*) realloc(arr, 10 * sizeof(int));
```

```
for (int i = 5; i < 10; i++) {  
    arr[i] = i + 1;
```

```
}
```

```
for (int i = 0; i < 10; i++) {  
    printf("%d", arr[i]);
```

```
}
```

```
free(arr); // deallocate memory
```

```
return 0;
```

```
}
```

```
free(arr); //
```

4) What is a dangling pointer?

def: A pointer that points to a memory location that has been freed or deallocated.

Cause: occurs when memory is deallocated using `free()` or `delete()` but pointer is not set to NULL.

Solution: Always set pointer to NULL after deallocation.

5) What is return value of `malloc()` if memory manager is unable to allocate the memory?

→ If `malloc()` fails to allocate memory, it returns NULL.

Ex:

```
int *ptr = (int*) malloc(10000000 * sizeof(int));
```

```
if (ptr == NULL) {
```

```
    printf("memory allocation failed. \n");
```

```
}
```


6] Write a program which is used to allocate memory for N floats dynamically using malloc(). Accept the value of N from user at runtime.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of floats: ");
```

```
    scanf("%d", &n);
```

```
    float *arr = (float *) malloc (n * sizeof (float));
```

```
    if (arr == NULL) {
```

```
        printf("Memory allocation failed. \n");
```

```
        return 1;
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        arr[i] = i * 1.1;
```

```
        printf("d0.2f", arr[i]);
```

```
    }
```

```
    free(arr); // Deallocate memory
```

```
    return 0;
```

```
}
```

7] Allocate dynamic memory for array of 5 elements where each element is of below structure type.

```
struct hello {
```

```
    float f;
```

```
    int i;
```

```
};
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```



```

struct hello {
    float f;
    int i;
};

```

```

int main() {
    struct hello *arr = (struct hello*) malloc(
        sizeof(struct hello));

    if (arr == NULL) {
        printf("memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        arr[i].f = i * 1.1;
        arr[i].i = i + 1;
        printf("Element %d: f = %.2f, i = %d\n",
            i, arr[i].f, arr[i].i);
    }

    free(arr); // Deallocate memory
    return 0;
}

```

- Explain internal working of new operator in detail.
- 1) New operator internally calls malloc()
 - 2) No need of typecasting in case of new operator
 - 3) No resized activity allowed in C++
i.e. No realloc()
 - 4) Initializes memory by calling the constructor (if for a class object).
 - 5) Returns a pointer to allocated memory.
- Internally uses operator new() for memory allocation
 - Operator new() can be overridden for custom behaviour

- Q1) What is need of typecasting for return value of malloc()?
- 1) malloc() returns a void*, which is a generic pointer
 2) typecasting is required to convert the void* into the appropriate data type.

Example:

```
int *ptr = (int *) malloc (size of int); // typecasting to int*
```

- Q2) Explain the working of below application, draw its diagrammatic representation & predict its output.
- 1) - ~~Enter~~ Stores no. of elements
 p - pointer to hold dynamically allocated memory
 iCnt is a loop counter
- 2) User prompted to enter no. of integers
- 3) - malloc() dynamically allocates memory for size integers
 - The (int*) typecasts the void* returned by malloc()
- 4) Input elements in array - Program stores size integers in dynamically allocated memory.
- 5) Printing entered elements - Program prints the values stored in allocated memory.
- 6) Freeing the allocated memory:
 free(p) deallocated dynamically allocated memory to avoid memory leaks.

Input

Enter number of elements.

3

Please enter the elements

10

20

30

Output

Entered elements are

10, 20, 30